

MASTER IMPLEMENTATION PROMPT FOR CURSOR

You are working as the **principal software engineer, machine learning engineer, research engineer, MLOps engineer, mobile architect, testing engineer, and technical research assistant** for the following academic research project:

PROJECT TITLE

Enhancing Clinical Decision Support through Explainable AI: Interpretable Machine Learning for Diabetes Risk Prediction

The final project must produce:

1. A rigorously evaluated **machine learning model suite** for diabetes risk prediction
2. An **Explainable AI layer** using SHAP and LIME
3. A working **mobile application**
4. A deployable **backend prediction API**
5. A reproducible ML experimentation pipeline
6. Model evaluation and explainability reports
7. A **research paper**
8. A Phase 1 **panel presentation**
9. Complete testing, documentation, reproducibility, deployment, and ethical analysis

This is a research project, not merely a software demo.

The implementation must therefore be:

- scientifically reproducible
- statistically defensible
- leakage-free
- modular
- well documented
- testable
- explainable
- safe for a healthcare-screening context
- suitable for eventual publication

- suitable for eventual mobile deployment

Do not produce a rushed notebook-only solution.

Build this as a proper research-grade software project.

0. REFERENCE MATERIALS AND SOURCE OF TRUTH

Before making changes, inspect every supplied project document in full.

Expected reference files include:

- Explainable_AI_Proposal_Document)_Reviewed V2(4).docx
- Explainable_AI_Presentation(8).pptx
- budget.xlsx
- any dataset files already present
- any existing code in the repository
- any README, notes, notebooks, or previous experiments

Do not assume the repository is empty.

First inspect it.

Do not delete or rewrite existing work without understanding it.

Latest supervisor instructions

The most recent project instructions are:

Final deliverables

- Mobile App
- AI Model
- Research Paper based on the project

Infrastructure guidance

AWS allows students to create accounts with credits.

Two accounts may potentially be available:

- NMIMS-domain account
- personal account

AWS may therefore be used for development, experimentation, deployment, or compute where useful.

Immediate Phase 1 requirement

1. Train and compare **five different machine learning algorithms**
2. Determine which performs best using:
 1. Accuracy
 2. Precision where appropriate
 3. Recall
 4. F1 Score
 5. ROC curve
 6. ROC-AUC
 7. and other scientifically relevant metrics already defined in the proposal
3. Draft a **research paper based on this work**
4. Prepare material for a **panel presentation by the first week of September**
5. Research paper must specifically include:
 1. ethical issues
 2. ethical concerns
 3. explainability concerns
 4. limitations of SHAP/LIME
 5. risks of misleading explanations

These latest instructions take priority for the immediate Phase 1 milestone.

1. IMPORTANT SOURCE RECONCILIATION

There are several differences between the long-term proposal and the latest supervisor instructions.

You must handle them deliberately rather than ignoring them.

1.1 Five algorithms now, ten algorithms eventually

The proposal defines the following ten-model suite:

1. Logistic Regression
2. Naive Bayes
3. K-Nearest Neighbours
4. Support Vector Machine
5. Random Forest
6. Gradient Boosting
7. XGBoost
8. LightGBM
9. CatBoost
10. Artificial Neural Network / Multilayer Perceptron

The current supervisor instruction asks for **five algorithms in Phase 1**.

Therefore:

- design the codebase to support all 10 algorithms
- make model registration configurable
- implement the common training/evaluation architecture so adding the remaining models later requires almost no architectural change

- run only the human-approved five algorithms for the official Phase 1 benchmark

VERY IMPORTANT

The documents do not clearly specify which exact five algorithms the supervisor currently wants.

Do not silently pretend they do.

Create:

`configs/phase1_models.yaml`

or equivalent configuration.

The five Phase 1 models must be controlled through this configuration.

If five algorithms have already been explicitly selected elsewhere in the repository, use those.

Otherwise:

1. implement the architecture necessary to support all candidate algorithms
2. document the ambiguity
3. prepare a scientifically reasonable suggested five-model set
4. mark it as a recommendation requiring confirmation
5. do not fabricate that the mentor selected it

A reasonable representative suggestion may span different model families, but it must remain clearly labelled as a proposed selection until confirmed.

2. ABSOLUTE RULE: NEVER FABRICATE

RESEARCH RESULTS

This rule is critical.

Never invent:

- model accuracies
- F1 scores
- recall values
- AUC values
- p-values
- confusion matrices
- training times
- explanation times
- SHAP rankings
- LIME rankings
- best-performing algorithms
- calibration scores
- dataset statistics that were not actually measured
- claims that one model outperformed another
- claims of clinical effectiveness
- claims of Indian-population performance

If experiments have not been executed successfully, write:

TBD - generated after experimental run

or equivalent.

The research paper must never contain fake experimental results simply to make the paper look complete.

Every numerical result appearing in the paper should ideally be generated programmatically from the experiment result files.

3. PROJECT GOAL

The project aims to build a mobile-first Explainable AI system that performs preliminary diabetes risk screening using self-reportable health indicators.

It must not simply answer:

What is the predicted risk?

It must additionally answer:

Why did the model arrive at this prediction?

The intended pipeline is:

User health information

- ☒ preprocessing
- ☒ trained ML model
- ☒ calibrated risk probability
- ☒ SHAP explanation
- ☒ LIME explanation
- ☒ explanation comparison/concordance
- ☒ safe human-readable explanation
- ☒ patient/doctor interface

The application is intended as a **screening aid**, not a diagnosis system.

That distinction must exist in:

- UI

- API
- model documentation
- README
- research paper
- presentation
- model card

Never describe the app as diagnosing diabetes.

4. DATASET

The proposal selects:

CDC BRFSS2015 Diabetes Health Indicators dataset

Expected characteristics from the project documents:

- approximately 253,680 cleaned records
- 21 health-indicator features
- self-reportable features
- no laboratory test required
- both sexes represented
- broad adult age range
- approximately 86:14 class imbalance
- binary diabetes-risk target in the selected cleaned dataset

The reason this dataset was chosen over PIMA is important to the research contribution:

PIMA requires laboratory-derived values such as glucose and insulin and is considerably smaller.

The project is intended to become a self-service mobile screening application, so a dataset whose inputs can be entered by users without laboratory tests is better aligned with the final product.

Do not guess the schema

Inspect the actual dataset.

Create a formal schema describing:

- exact feature names
- data types
- allowed ranges
- categorical mappings
- target column
- class definitions
- missing-value expectations

Create something such as:

`configs/data_schema.yaml`

or:

`src/data/schema.py`

Do not hardcode feature names based purely on memory if they can be derived from the actual source dataset.

5. DATASET REPRODUCIBILITY

Create a complete data-loading pipeline.

Suggested structure:

`src/`

data/
load.py
validate.py
preprocess.py
split.py
schema.py

The data pipeline should:

1. load the original dataset
2. validate expected columns
3. validate target labels
4. check duplicate rows
5. audit missing values
6. inspect distributions
7. identify invalid values
8. create reproducible train/validation/test splits
9. save split metadata
10. calculate and save dataset statistics

Never silently repair corrupted inputs.

If data validation fails, produce a clear error.

6. STRICT ANTI-LEAKAGE METHODOLOGY

This is one of the most important scientific requirements.

The proposal requires approximately:

- Train: 70%
- Validation: 15%
- Test: 15%

with stratification on the diabetes label.

Use a fixed random seed.

Save the indices or stable row identifiers representing each split so the exact same split can be reproduced.

Example:

```
artifacts/splits/  
train_indices.csv  
validation_indices.csv  
test_indices.csv  
split_metadata.json
```

The test set is sacred

The held-out test set must never be used for:

- scaler fitting
- imputation fitting
- SMOTE
- feature selection
- hyperparameter tuning
- threshold tuning
- model selection
- calibration fitting
- exploratory model comparison during development

It should be evaluated **only after model configuration is frozen**.

7. SMOTE: IMPLEMENT WITHOUT LEAKAGE

The source documents contain a visual ordering ambiguity, but the detailed methodology explicitly states that SMOTE belongs only in training.

Therefore:

Correct implementation

Split first.

Then apply SMOTE only inside training.

For cross-validation, SMOTE must happen **inside each training fold**, not once globally before CV.

Use an `imblearn.pipeline.Pipeline` or equivalent so the sequence is:

fold training data

▣ preprocessing fitted on fold training data

▣ SMOTE on fold training data

▣ model training

▣ validation on untouched fold validation data

Never do:

`SMOTE(full_dataset)`

`train_test_split(...)`

Never do:

`SMOTE(train)`

`cross_val_score(...)`

if the resampled dataset is shared across CV folds.

That leaks information.

Write automated tests specifically checking that resampling does not touch validation/test observations.

8. PREPROCESSING

Inspect the actual dataset before deciding transformations.

Do not unnecessarily one-hot encode values already represented correctly.

However, algorithm-specific preprocessing should be handled properly.

For example, algorithms dependent on scale may require scaling:

- Logistic Regression
- KNN
- SVM
- ANN/MLP

Tree-based methods typically do not require standard scaling:

- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

Create algorithm-specific pipelines so every model receives appropriate preprocessing without creating unfair comparisons.

Store preprocessing together with each model artifact.

Do not save a classifier without the preprocessing required to reproduce its prediction.

9. MISSING VALUES

The reviewed proposal states that the cleaned dataset is expected to contain no missing values.

Still:

- run a real missing-value audit
- log the results
- never simply assume zero missing values

If a defensive imputation pipeline is retained, it must:

- be fitted only on training data
- never learn values from validation/test data
- be serialized with the model

Produce:

reports/data_quality_report.md

with:

- record count
- feature count
- target distribution
- duplicates
- missing values
- range checks
- invalid values
- split statistics

10. OUTLIER HANDLING

The proposal discusses retaining all 21 features and handling BMI outliers without indiscriminately deleting observations.

If the repository implementation includes IQR-based capping:

- calculate thresholds from training data only
- apply those learned thresholds to validation/test
- serialize them as preprocessing metadata
- compare results with and without capping if practical

Do not remove medically unusual values simply because they are statistically uncommon unless there is strong justification.

Distinguish:

- impossible value
 - data-entry error
 - valid extreme observation
-

11. PHASE 1 MODEL BENCHMARKING ARCHITECTURE

Create one common interface for models.

Example concept:

```
class ModelDefinition:
    name
    estimator
    preprocessing
    parameter_space
    tuning_strategy
    supports_predict_proba
    explainability_strategy
```

Models should be registered centrally rather than hardcoded across separate scripts.

Suggested structure:

```
src/  
models/  
  registry.py  
  logistic_regression.py  
  naive_bayes.py  
  knn.py  
  svm.py  
  random_forest.py  
  gradient_boosting.py  
  xgboost_model.py  
  lightgbm_model.py  
  catboost_model.py  
  mlp.py
```

A central command should allow:

```
python -m src.training.train --config configs/phase1_models.yaml
```

Do not create ten completely unrelated notebooks with copied training code.

12. REPRODUCIBILITY

Set and record random seeds for:

- Python
- NumPy
- scikit-learn
- SMOTE
- Optuna where applicable
- XGBoost
- LightGBM
- CatBoost
- PyTorch/TensorFlow if used

Record:

- Python version
- package versions
- OS information

- CPU/GPU information where useful
- dataset hash
- git commit hash
- configuration file
- random seed
- experiment timestamp

Create:

artifacts/experiments/<experiment_id>/metadata.json

13. HYPERPARAMETER TUNING

The proposal specifically discusses:

- 5-fold stratified cross-validation
- Bayesian optimisation for heavier models
- Optuna
- lighter search for simpler algorithms

Implement hyperparameter optimisation modularly.

For expensive algorithms such as:

- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost
- ANN

support Optuna-based tuning.

For simpler models, use an appropriate small search strategy rather than wasting compute.

Do not run extremely large hyperparameter searches simply because they are possible.

Track:

- trial number
- parameters
- CV score
- runtime
- failure reason
- best parameters
- seed

Make the optimisation metric configurable.

Because this is an imbalanced healthcare-screening problem, do not automatically optimise raw accuracy.

A likely primary optimisation objective should be selected deliberately based on Phase 1 goals, such as F1 or recall, while preserving all required metrics.

Record the choice and reasoning.

14. EVALUATION METRICS

For every Phase 1 model, calculate at minimum:

- Accuracy
- Precision
- Recall / Sensitivity
- F1 Score
- Specificity
- Confusion Matrix
- ROC Curve

- ROC-AUC

Also calculate the proposal's additional calibration metrics where technically applicable:

- Brier Score
- Expected Calibration Error
- Calibration curve

Because the dataset is imbalanced, also consider:

- Precision-Recall curve
- PR-AUC / Average Precision

Treat this as an additional imbalance diagnostic, not a replacement for the requested ROC-AUC.

15. FALSE NEGATIVES MATTER

This is healthcare screening.

A false negative means someone actually belonging to the risk-positive class is predicted as negative.

The paper must therefore not select a model purely because it has the highest overall accuracy.

For example, if:

Model A

Accuracy: 90%

Recall: 50%

Model B

Accuracy: 87%

Recall: 82%

Model A cannot automatically be declared superior.

The evaluation framework must discuss tradeoffs among:

- sensitivity/recall
- specificity
- precision
- F1
- calibration
- false-negative burden
- decision threshold

Do not choose the production model using one metric in isolation.

16. THRESHOLD HANDLING

Default classifier threshold 0.5 must not automatically be treated as medically optimal.

For Phase 1:

- report performance at default threshold
- optionally evaluate threshold tradeoffs on the validation set
- never optimise a threshold using test data

If a threshold is tuned, document:

- target metric
- selected threshold
- validation result
- test result
- rationale

Avoid claiming a threshold has clinical validity unless actual clinical validation exists.

17. CALIBRATION

The mobile app intends to show a risk percentage.

Therefore probability quality matters.

A classifier outputting 0.73 is not useful as a user-facing "73% risk" unless probability calibration is reasonably validated.

Evaluate calibration using:

- Brier score
- calibration plots
- ECE where implemented

For poorly calibrated models, consider:

- Platt scaling
- isotonic calibration
- another defensible calibration method

Calibration must only be fitted using non-test data.

The held-out test set remains untouched.

18. MODEL COMPARISON OUTPUTS

Automatically generate:

```
results/  
phase1/
```

model_comparison.csv
model_comparison.json
test_metrics.json
roc_curves.png
precision_recall_curves.png
calibration_curves.png
training_times.csv
inference_times.csv
confusion_matrices/
hyperparameters/
experiment_summary.md

model_comparison.csv should have fields such as:

model
accuracy
precision
recall
specificity
f1
roc_auc
pr_auc
brier_score
ece
cv_mean
cv_std
training_time
inference_time
decision_threshold
random_seed

Only include actual measured values.

19. STATISTICAL STABILITY

Do not report only a single metric without uncertainty.

For CV metrics, save:

- mean
- standard deviation
- individual fold values

Where practical, calculate confidence intervals for important held-out test metrics using a defensible bootstrap procedure.

Do not manufacture statistical significance.

If comparing models statistically, clearly document the method and assumptions.

20. FINAL PHASE 1 MODEL SELECTION

Generate a model-selection report.

The report should compare the five approved algorithms based on:

1. Recall
2. F1
3. ROC-AUC
4. Precision
5. Specificity
6. Calibration
7. inference latency
8. explainability compatibility
9. computational cost
10. stability across folds

Do not automatically write:

XGBoost is the best model.

The proposal identifies boosting models as strong candidates, but the project must experimentally determine the winner.

The actual data decides.

21. MODEL ARTIFACT FORMAT

For each trained model save:

```
artifacts/models/<model_name>/  
  model.*  
  pipeline.*  
  config.yaml  
  metrics.json  
  feature_schema.json  
  training_metadata.json  
  model_card.md
```

For the selected production model additionally create:

```
artifacts/production/  
  model.*  
  pipeline.*  
  feature_schema.json  
  model_metadata.json  
  calibration_metadata.json  
  decision_threshold.json  
  model_card.md  
  version.json
```

The API must later load one complete reproducible inference artifact.

22. EXPLAINABILITY LAYER

The proposal requires:

SHAP

Used for:

- global explanation
- local explanation
- feature importance
- doctor/audit view

LIME

Used primarily for:

- local explanation
- independent local surrogate comparison

Implement XAI only after a valid model evaluation pipeline exists.

Do not start with visualisations while model training methodology remains unreliable.

23. SHAP IMPLEMENTATION

Use the most appropriate SHAP explainer based on model family.

For tree-based production models prefer TreeSHAP when valid.

Generate:

Global explanations

- feature importance
- mean absolute SHAP values
- summary plot
- beeswarm plot where appropriate

Local explanations

For individual predictions:

- feature contributions
- contribution direction
- baseline
- model output
- waterfall-style output
- ranked features

Save machine-readable values as JSON in addition to plots.

Never make the mobile/API depend on parsing an image.

24. LIME IMPLEMENTATION

For representative samples and eventually individual predictions:

- generate local explanations
- rank influential features
- record positive/negative contributions
- record explanation runtime
- record LIME configuration and random seed

Because LIME is stochastic, test its stability.

Run repeated LIME explanations for selected observations and quantify how often the ranked factors change.

This should feed directly into the ethical discussion.

25. SHAP/LIME AGREEMENT

The proposal contains an important project contribution:

Comparing whether SHAP and LIME identify similar top factors.

Implement this explicitly.

For each evaluated prediction generate:

```
{  
  "shap_top_features": [],  
  "lime_top_features": [],  
  "top_k": 3,  
  "exact_top_k_set_match": false,  
  "overlap_count": 0,  
  "jaccard_overlap": 0.0  
}
```

Possible aggregate metrics:

- exact top-3 agreement rate
- mean overlap
- mean Jaccard score
- directional agreement
- rank correlation where meaningful

However:

CRITICAL ETHICAL LANGUAGE

Do **not** call SHAP/LIME agreement "model certainty".

Do not imply:

SHAP and LIME agree, therefore the prediction is correct.

They measure explanations using different methods.

Agreement means **explanation-method concordance**, not validated predictive confidence.

You may expose an "explanation agreement" or "explanation concordance" indicator.

The paper must explicitly discuss this limitation.

If the original proposal uses the phrase "confidence indicator", preserve that as the original project concept but explain scientifically that it is confidence in explanation consistency, not calibrated predictive confidence.

26. EXPLANATIONS DO NOT ESTABLISH CAUSALITY

This must appear prominently in:

- research paper
- model card
- application design
- doctor view
- patient view documentation

For example:

If high BMI receives a large positive SHAP contribution, this means the trained model relied heavily on that feature for this prediction.

It does **not** prove that BMI caused this individual's disease risk.

Similarly:

- SHAP is not causal inference
- LIME is not causal inference
- feature importance is not causal effect
- explanation agreement does not establish truth

Never use causal language unless causality was actually established.

27. EXPLANATION SAFETY

The proposal suggests converting top features into understandable suggestions.

Do not ask a generative AI system to invent medical advice dynamically.

Implement a deterministic explanation/template layer.

For example conceptually:

feature
direction
patient_safe_description
modifiable
clinical_review_status

Templates must be version-controlled.

Example categories:

- modifiable factor
- partially modifiable factor
- non-modifiable factor

If a template has not received clinical review, mark:

clinical_review_status: pending

Do not pretend that an engineer-authored recommendation is clinically approved.

28. HEALTHCARE ETHICS SECTION

The supervisor specifically requires ethical issues and concerns involving explainability.

Create a dedicated research section covering at least:

28.1 False reassurance

A false-negative prediction could cause a user to incorrectly believe they have no reason to seek medical evaluation.

28.2 Unnecessary anxiety

A false-positive screening result may create unnecessary concern.

28.3 Explanation illusion

A visually convincing SHAP or LIME explanation can make an imperfect prediction appear more

authoritative than it actually is.

28.4 Correlation versus causation

Feature attribution is not causal evidence.

28.5 Explanation instability

LIME can vary because of perturbation and random sampling.

SHAP results can also depend on model/background/reference assumptions.

28.6 Explanation disagreement

SHAP and LIME may highlight different features.

Do not hide disagreement.

28.7 Automation bias

Patients or clinicians may over-trust a machine-generated score because an explanation is provided.

28.8 Dataset shift

BRFSS2015 represents a US population.

Performance in India cannot be assumed.

28.9 Population representation

Risk relationships may differ across:

- ethnicity
- geography
- age groups
- sex
- socioeconomic groups

28.10 Self-reported information

User-reported answers may contain:

- recall errors
- misunderstanding
- intentional misreporting
- inconsistent interpretation

28.11 Class imbalance

Reported accuracy may hide poor detection of the minority class.

28.12 Explainability versus privacy

Explanations can reveal sensitive health-related information.

28.13 Doctor versus patient communication

Raw SHAP values may be inappropriate for patients without context.

28.14 Non-diagnostic nature

The system is a preliminary screening aid.

28.15 Human oversight

No medical decision should be made solely from the model output.

29. FAIRNESS AND SUBGROUP ANALYSIS

Where dataset variables permit it, calculate subgroup performance.

Potential groupings may include existing demographic variables such as:

- sex
- age group
- other defensible demographic attributes already present

Only use variables actually present in the dataset.

For each subgroup consider:

- sample size
- prevalence
- recall
- precision
- specificity
- F1
- ROC-AUC
- false-negative rate
- calibration

Do not make strong fairness claims when subgroup sample sizes are insufficient.

Report limitations clearly.

30. US DATA VERSUS INDIAN USERS

This is a major limitation.

The production dataset originates in the US.

Therefore:

- never claim Phase 1 performance represents Indian clinical performance
- never label the score as clinically validated for India
- make transportability limitations explicit

The proposal includes later Indian validation and adaptation.

Keep architecture ready for that future phase.

Potential later sources discussed in the proposal include Indian cohort data such as ICMR-INDIAB or NFHS-related research, subject to legitimate data access and compatible feature definitions.

Never fabricate access to those datasets.

Never create synthetic "Indian validation results" and report them as real.

31. SOUTH-ASIAN VALIDATION

The proposal also mentions possible early validation using the Early Stage Diabetes Risk Prediction dataset associated with Sylhet, Bangladesh.

If this is eventually implemented:

- first inspect feature compatibility
- do not directly compare incompatible features as though datasets were identical
- document mapping assumptions
- treat it only as a supplementary external-validation exercise

It cannot replace Indian population validation.

32. RESEARCH PAPER WORKFLOW

Create:

```
research/  
  paper/  
    README.md  
    outline.md  
    draft.md  
    references.bib  
    figures/  
    tables/  
    generated_results/
```

The paper should ultimately contain:

Title

Enhancing Clinical Decision Support through Explainable AI: Interpretable Machine Learning for Diabetes Risk Prediction

Proposed paper structure

1. Abstract
2. Keywords
3. Introduction

4. Motivation and Clinical Context
 5. Problem Statement
 6. Research Gap
 7. Related Work / Literature Review
 8. Contributions
 9. Dataset
 10. Data Preprocessing
 11. Experimental Methodology
 12. Machine Learning Algorithms
 13. Hyperparameter Optimisation
 14. Evaluation Metrics
 15. Experimental Results
 16. Model Comparison
 17. Explainable AI Methodology
 18. SHAP Analysis
 19. LIME Analysis
 20. SHAP-LIME Explanation Concordance
 21. Calibration Analysis
 22. Discussion
 23. Ethical Issues and Explainability Concerns
 24. Fairness and Bias
 25. Limitations
 26. Applicability to Indian Population
 27. Mobile Deployment Architecture
 28. Future Validation
 29. Conclusion
 30. References
-

33. PHASE 1 PAPER

The September Phase 1 paper draft does not need to pretend the full mobile system is finished.

Its main contribution should focus on:

- dataset
- preprocessing
- five-model benchmark
- reproducible methodology
- evaluation

- ROC analysis
- F1
- recall
- model comparison
- explainability strategy
- preliminary SHAP/LIME work if complete
- explainability ethics
- limitations
- planned mobile deployment

Sections dependent on future phases can be labelled clearly as:

- proposed implementation
- future work
- planned validation

Never write future work in past tense.

34. LITERATURE

The reference documents already contain a literature survey.

Preserve and organise those references.

Relevant referenced areas include work on:

- diabetes prediction with SHAP/LIME
- Logistic Regression
- Random Forest
- XGBoost
- LightGBM
- CatBoost
- neural networks
- SHAP
- LIME
- tabular ML

- BRFSS
- Indian diabetes data
- healthcare AI regulation

Do not invent papers.

If bibliographic metadata is incomplete or contradictory, mark it for verification.

Do not create imaginary DOIs.

35. AUTOMATED PAPER TABLES

The paper's model-results table should ideally be generated directly from:

results/phase1/model_comparison.csv

For example:

Model | Accuracy | Precision | Recall | F1 | ROC-AUC | Brier Score

Do not manually copy values into several files if an automated generator can prevent transcription mistakes.

Create a script such as:

```
python -m src.reporting.generate_paper_assets
```

that produces:

- model table
- ROC figure
- confusion matrices
- calibration plots
- selected SHAP plots
- dataset distribution plots

36. PHASE 1 PRESENTATION SUPPORT

Create:

```
presentation/  
  phase1/  
    panel_outline.md  
    slide_content.md  
    speaker_notes.md  
    figures/  
    key_numbers.json
```

Prepare content for approximately:

1. Title
2. Problem
3. Why diabetes
4. Research gap
5. Dataset selection
6. Methodology
7. Five models
8. Experimental setup
9. Evaluation metrics
10. Results
11. ROC comparison
12. Best model and tradeoffs
13. Explainability
14. Ethical concerns
15. Limitations
16. Next phase
17. Conclusion

All result numbers must be pulled from real experiment outputs.

37. MOBILE APPLICATION ARCHITECTURE

The reviewed proposal specifies:

- Flutter
- Android and iOS
- FastAPI backend
- Firebase authentication
- prediction history
- patient view
- doctor view

Preserve this overall architecture unless the team explicitly approves a change.

Proposed system:

Flutter App

|

| HTTPS

v

FastAPI Backend

|

+--> Input validation

|

+--> ML preprocessing pipeline

|

+--> Production model

|

+--> Calibration / threshold

|

+--> SHAP

|

+--> LIME

|

+--> Safe explanation mapper

|

v

Structured prediction response

Authentication and history can remain Firebase-based as proposed.

AWS availability does not automatically mean Firebase must be replaced.

Do not unnecessarily redesign an approved architecture.

38. API CONTRACT

Eventually implement endpoints such as:

GET /health
GET /model/info
POST /predict
POST /explain
GET /predictions/{id}
GET /user/history
Exact routes can evolve.

A prediction request should validate all required features.

Example response architecture:

```
{
  "prediction": {
    "class": "higher_risk",
    "probability": 0.0,
    "threshold": 0.0
  },
  "explanation": {
    "shap": {
      "top_features": []
    },
    "lime": {
      "top_features": []
    },
    "concordance": {
      "top_k": 3,
      "overlap": 0,
      "agreement_level": ""
    }
  }
}
```

```
}  
},  
"disclaimer": "",  
"model": {  
  "version": "",  
  "trained_population": "CDC BRFSS 2015 US population"  
}  
}
```

Do not use this exact risk wording unless approved by the app UX and research team.

39. API INPUT VALIDATION

Every field should have:

- data type
- valid range/categories
- human-readable label
- missing-value behaviour
- mapping from mobile value to model encoding

Invalid values should return HTTP 422 with understandable field errors.

Never silently coerce impossible health inputs into a valid range without logging it.

40. MOBILE PATIENT VIEW

The patient interface should prioritise simplicity.

Potential content hierarchy:

1. Screening result
2. Risk probability if scientifically justified and calibrated

3. Main contributing factors
4. Explanation agreement/disagreement
5. Short safe explanation
6. Screening disclaimer
7. recommendation to consult a healthcare professional where appropriate

Avoid:

- raw SHAP values without context
- scary red alerts without medical justification
- phrases such as "you have diabetes"
- deterministic medical claims

41. MOBILE DOCTOR / TECHNICAL VIEW

The proposal expects a richer clinician-facing view.

Possible components:

- model probability
- classification threshold
- ranked feature contributions
- SHAP waterfall
- SHAP global context
- LIME ranking
- explanation concordance
- feature values
- model version
- limitations
- population context
- calibration information

Do not imply it is clinically approved merely because it has a doctor view.

42. RESPONSE-TIME TARGETS

The proposal defines approximately:

- model inference under 200 ms
- combined SHAP/LIME explanation under 1.5 seconds
- end-to-end target under 2 seconds

Build a proper benchmark later.

Measure:

- warm inference
- cold inference
- SHAP
- LIME
- serial SHAP+LIME
- potentially parallel SHAP/LIME
- API overhead
- network-independent server time

Do not claim the target is achieved until measured.

Generate:

reports/latency_benchmark.md

43. EXPLAINABILITY PERFORMANCE

Explanation latency is itself part of the research contribution.

Track:

model inference latency
SHAP latency
LIME latency
total explanation latency
total server latency

Report:

- median
- p95
- p99 where sample count is sufficient

If LIME prevents the 2-second target, document the measured tradeoff rather than hiding it.

Potential architecture changes may later include:

- asynchronous doctor-level explanation
- caching
- TreeSHAP
- reducing LIME perturbations

But do not implement shortcuts that reduce explanation quality without measuring the effect.

44. CLOUD STRATEGY

The latest supervisor guidance allows AWS student credits.

The budget workbook contains cloud-related planning from an earlier architecture.

Do not assume every older GCP-specific budget item remains a hard technical requirement.

At the same time, do not rewrite approved budget documents without permission.

Build the application so the core system is cloud-portable.

Phase 1

Training must work:

- locally
- and optionally on AWS compute

Later deployment

FastAPI must be Dockerized so deployment can move to an appropriate AWS service without code rewrites.

Do not tightly couple inference logic to AWS SDKs.

45. AWS ACCOUNT SECURITY

Very important:

Even if an AWS **root account** is created for the student account, root credentials must never be used as normal application/development credentials.

Document:

1. Enable MFA on root.
2. Do not create root access keys.
3. Create IAM identities/roles for development.
4. Follow least privilege.
5. Store secrets outside Git.
6. Configure AWS budget alarms.
7. Configure billing alerts.

8. Never commit:

1. AWS access key
2. secret access key
3. Firebase secrets
4. API keys
5. service account JSON

9. Use environment variables/secrets management.

Create:

docs/aws_setup.md

Do not automatically provision paid AWS infrastructure without explicit confirmation.

46. CLOUD COST CONTROL

The supervisor mentioned approximately \$200 student credits/account.

Do not interpret credits as unlimited budget.

Add documentation for:

- checking current balance
- stopping unused instances
- avoiding idle GPU instances
- instance termination
- storage cleanup
- budget alarms
- per-service cost visibility

For Phase 1, CPU-based algorithms should be tested locally before using expensive cloud compute.

Do not provision GPUs simply because it sounds like an AI project.

Most of the tabular algorithms may not need GPU acceleration.

47. REPOSITORY STRUCTURE

Unless an existing repository structure strongly suggests otherwise, target something similar to:

```
explainable-diabetes-ai/
├──
├── README.md
├── LICENSE
├── .gitignore
├── .env.example
├── pyproject.toml
├── requirements/
├──
├── configs/
├──   ├── base.yaml
├──   ├── phase1_models.yaml
├──   ├── data_schema.yaml
├──   └── experiments/
├──
├── data/
├──   ├── README.md
├──   ├── raw/
├──   ├── processed/
├──   └── splits/
├──
├── src/
├──   ├── data/
├──   ├── preprocessing/
├──   ├── models/
├──   ├── training/
├──   ├── evaluation/
├──   ├── calibration/
├──   ├── explainability/
├──   ├── reporting/
├──   └── utils/
```

```

├── artifacts/
│   ├── models/
│   ├── production/
│   ├── splits/
│   ├── experiments/
│   └──
├── results/
│   ├── phase1/
│   └──
├── notebooks/
│   ├── exploration/
│   └──
├── api/
│   ├── app/
│   ├── tests/
│   ├── Dockerfile
│   └──
├── mobile/
│   ├── flutter_app/
│   └──
├── research/
│   ├── paper/
│   └──
├── presentation/
│   ├── phase1/
│   └──
├── reports/
│   └──
├── infra/
│   ├── aws/
│   └──
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── leakage/
│   └──
├── docs/
│   ├── architecture.md
│   ├── methodology.md
│   ├── source_of_truth.md
│   ├── ethics.md
│   ├── aws_setup.md
│   └── experiment_protocol.md

```

Adapt this if a good repository structure already exists.

Do not reorganise existing code unnecessarily.

48. NOTEBOOK POLICY

Jupyter notebooks may be used for exploration and visual analysis.

They must not become the only implementation.

Reusable logic belongs in Python modules.

For example, do not leave model training only in:

`notebooks/final_model_v7_REAL_FINAL.ipynb`

The actual pipeline should be runnable from commands/scripts.

Notebooks should import project modules.

49. COMMAND-LINE WORKFLOW

Create clean commands for important operations.

Examples:

```
python -m src.data.validate  
python -m src.data.prepare
```

```
python -m src.training.train --config configs/phase1_models.yaml
python -m src.evaluation.evaluate
python -m src.explainability.generate
python -m src.reporting.generate_phase1_report
```

A developer should not need to manually run 40 notebook cells in a particular order.

50. LOGGING

Use structured logging.

Important events:

- dataset loaded
- data hash
- split created
- model started
- model completed
- model failed
- tuning started
- tuning completed
- best parameters
- metrics calculated
- artifact saved
- SHAP started
- LIME started
- report created

Avoid printing uncontrolled massive objects.

51. FAILURE HANDLING

Training one model should not destroy every other result.

For example, if CatBoost fails:

- record failure
- save logs
- allow other configured algorithms to complete
- return a clear final summary

Do not silently skip failed experiments.

The research report must distinguish:

- completed
 - failed
 - not run
-

52. TEST SUITE

Testing is mandatory.

Data tests

Test:

- required columns exist
- target values valid
- no split overlap
- expected dimensions
- schema works

Leakage tests

Test:

- train/validation/test indices are disjoint
- preprocessing fits only on training data
- SMOTE never sees validation/test
- test data is untouched during tuning

Metric tests

Use small known arrays to verify:

- precision
- recall
- specificity
- F1
- ROC-AUC
- confusion matrix calculations

Serialization tests

Train tiny model, save it, reload it, verify prediction consistency.

Explainability tests

Verify SHAP/LIME output:

- valid feature names
- expected length
- serializable JSON
- no NaN where not expected

API tests later

- valid request
- missing field
- invalid category
- out-of-range input
- model unavailable

- successful prediction

53. MODEL CARD

Create a model card for every final candidate.

At minimum:

Model name

Model version

Algorithm

Dataset

Dataset population

Features

Target

Train/validation/test split

Training date

Random seed

Hyperparameters

Evaluation metrics

Calibration

Known limitations

Intended use

Out-of-scope use

Fairness considerations

Explainability method

Ethical considerations

Deployment requirements

Production model card must state clearly:

Not a diagnostic tool.

54. DATA CARD

Create:

docs/data_card.md

Include:

- source
 - year
 - population
 - sample count
 - feature count
 - target
 - class imbalance
 - self-reported nature
 - known geographic limitation
 - intended research use
 - relevant preprocessing
 - limitations
 - why BRFSS was chosen over PIMA
-

55. PRIVACY

Do not collect more information than necessary.

For the future mobile application:

- do not log raw health responses unnecessarily
- do not place health data in debug logs
- do not send prediction history to analytics tools without explicit design approval
- use HTTPS
- authenticate history access
- separate user identity from model-processing logic where practical
- establish retention rules before real pilot use

Do not claim HIPAA compliance, GDPR compliance, or Indian healthcare-law compliance unless

it has actually been assessed.

56. FIREBASE

The proposal currently uses Firebase for authentication and risk-history sync.

Keep secrets/configuration environment-specific.

Do not commit service credentials.

Architect data records with version metadata such as:

prediction_id
user_id
created_at
model_version
schema_version
risk_output
explanation_version

Do not store unnecessary raw sensitive features indefinitely without an approved retention strategy.

57. VERSIONING

Version:

- dataset
- split
- model
- preprocessing pipeline
- explanation templates

- API schema

Example:

```
dataset_version: brfss2015-clean-v1
split_version: split-v1
model_version: phase1-xgb-v1
explanation_version: xai-v1
api_version: v1
```

A saved prediction should identify which model produced it.

58. PHASE-WISE DEVELOPMENT PLAN

The entire project should be implemented progressively.

Do not build everything simultaneously.

PHASE 0: REPOSITORY AUDIT AND RESEARCH CONTRACT

Before coding significantly:

Tasks

1. Read every reference document.
2. Inspect repository.
3. Inspect dataset availability.
4. Inspect installed dependencies.
5. Identify incomplete work.
6. Identify inconsistencies among proposal, slides, budget, and newest supervisor instructions.

7. Create:

1. docs/source_of_truth.md
2. docs/implementation_plan.md
3. docs/experiment_protocol.md
4. docs/ethics.md

8. Define immediate September milestone.

9. Define long-term project phases.

10. Ensure code architecture can support all ten algorithms.

Deliverable

A short audit report containing:

Existing state

Missing items

Source inconsistencies

Immediate Phase 1 deliverables

Implementation sequence

Blocking decisions

Do not spend days only writing plans.

After the audit, proceed directly into Phase 1 implementation.

PHASE 1A: DATA PIPELINE

Implement

- data loading
- schema validation
- missing-value audit
- class distribution
- duplicate audit
- reproducible split
- preprocessing
- training-only SMOTE

- data-quality report

Acceptance criteria

- dataset loads reproducibly
 - exact split can be recreated
 - no split overlaps
 - test fold remains untouched
 - data quality report generated
 - tests pass
-

PHASE 1B: FIVE-MODEL EXPERIMENT FRAMEWORK

Implement

- configurable model registry
- five approved Phase 1 algorithms
- algorithm-specific preprocessing
- CV
- tuning
- metric calculation
- experiment metadata
- persistence
- failure tracking

Acceptance criteria

Each model can run from the same experiment command and generate a standard metrics file.

PHASE 1C: MODEL EVALUATION

Generate:

- comparison table
- ROC plots
- confusion matrices
- precision-recall plots
- calibration results
- CV results
- held-out test results
- training time
- inference time

Produce:

results/phase1/experiment_summary.md

Include:

- best candidate by each metric
- tradeoffs
- errors/failures
- uncertainty
- no fabricated conclusions

PHASE 1D: INITIAL EXPLAINABILITY

For the leading candidate(s):

- generate global SHAP
- local SHAP
- local LIME
- explanation runtime
- top-k agreement
- repeated LIME stability test
- machine-readable explanation output

Do not build the full patient recommendation engine yet if clinical templates are not reviewed.

PHASE 1E: ETHICAL ANALYSIS

Create a detailed ethics section covering:

- bias
 - fairness
 - US-to-India generalisation
 - self-reported inputs
 - false negatives
 - false positives
 - calibration
 - misleading probability interpretation
 - explanation instability
 - SHAP limitations
 - LIME limitations
 - causality misconception
 - automation bias
 - over-trust
 - privacy
 - human oversight
 - clinical validation gap
 - non-diagnostic usage
 - explanation concordance versus prediction confidence
-

PHASE 1F: RESEARCH PAPER DRAFT

Generate a complete draft with:

- finished methodology
- actual Phase 1 results
- figures

- tables
- explainability section
- ethics section
- limitations
- future work

If experiments are still running, leave explicit placeholders.

Never fill them with fictional values.

PHASE 1G: PANEL PRESENTATION PACKAGE

Generate:

- key figures
- concise results table
- architecture diagram specification
- methodology diagram
- ROC comparison
- SHAP example
- ethical issues slide
- next-phase roadmap
- speaker notes

The first-week-of-September presentation must focus on demonstrable work rather than promised features.

PHASE 2: FULL TEN-MODEL BENCHMARK

After Phase 1 approval:

Extend the same framework to all ten algorithms:

1. Logistic Regression
2. Naive Bayes
3. KNN
4. SVM
5. Random Forest
6. Gradient Boosting
7. XGBoost
8. LightGBM
9. CatBoost
10. ANN/MLP

Do not create a second unrelated experiment system.

Reuse Phase 1 infrastructure.

Run:

- tuning
- CV
- held-out evaluation
- calibration
- latency
- explainability compatibility

Select final production candidate scientifically.

PHASE 3: COMPLETE EXPLAINABILITY ENGINE

Implement production-quality:

- SHAP
- LIME
- explanation concordance
- explanation latency
- explanation schema

- safe feature-to-text mapping
- patient explanation
- doctor explanation
- XAI stability reporting

Perform explainability evaluation on sufficiently large representative samples.

PHASE 4: FASTAPI BACKEND

Build:

- health endpoint
- model-info endpoint
- prediction endpoint
- explanation endpoint if separate
- Pydantic schemas
- model lifecycle loading
- error handling
- authentication integration
- logging
- latency instrumentation
- Dockerfile
- OpenAPI documentation
- tests

Production model should load once, not on every request.

PHASE 5: FLUTTER MOBILE APP

Build:

Authentication

- signup/login
- session handling

Questionnaire

- all required BRFSS-compatible inputs
- plain-language field descriptions
- validation
- progress state

Patient results

- risk output
- top factors
- safe explanation
- explanation concordance
- disclaimer

Doctor/technical view

- ranked contributions
- SHAP visualisation
- LIME output
- model metadata
- population context

History

- previous screenings
- timestamp
- model version

Keep state management and API layers separated from visual widgets.

PHASE 6: AWS DEPLOYMENT

Once Phase 1 and API architecture are stable:

- containerize API
- select an appropriate AWS hosting method
- configure HTTPS
- configure secure secrets
- configure logs
- configure monitoring
- configure budget alerts
- deploy staging
- run API integration tests

Do not deploy expensive infrastructure unnecessarily.

PHASE 7: INDIAN VALIDATION AND ADAPTATION

Only when legitimate compatible data becomes available:

1. Freeze BRFSS model.
2. Evaluate unchanged model on external Indian dataset.
3. Measure performance degradation.
4. Measure calibration shift.
5. Compare subgroup metrics.
6. Recompute SHAP importance.
7. Compare feature-ranking changes.
8. Recalibrate where justified.
9. Fine-tune/retrain only if methodologically justified.
10. Document before/after results.

Do not merge Indian validation data into the test process carelessly.

Maintain a separate external-validation protocol.

PHASE 8: FINAL TESTING AND PILOT

Perform:

- unit tests
- integration tests
- model tests
- API tests
- mobile tests
- latency/load tests
- security review
- explanation consistency tests
- usability review
- model-version verification
- privacy review

Pilot only after disclaimers and data handling have been reviewed.

PHASE 9: FINAL PAPER

Update the Phase 1 paper with:

- full ten-model benchmark
- final production model
- full XAI results
- mobile implementation
- system latency
- Indian validation if available
- ethics
- limitations
- deployment
- future work

Never claim external clinical validation that was not performed.

59. README REQUIREMENTS

The root README must explain:

What the project is

Explainable diabetes risk screening using ML + SHAP + LIME.

Current status

Example:

Phase 1 - five-model benchmark in progress

Do not claim future phases are complete.

Setup

- Python version
- virtual environment
- dependencies
- dataset placement
- configuration
- training command
- evaluation command

Research methodology

Brief description of:

- split
- CV
- SMOTE
- metrics
- held-out testing

Safety

Prominent statement:

This project is an experimental screening/research system and is not a medical diagnostic tool.

60. CODING QUALITY

Use:

- type hints
- docstrings where useful
- dataclasses/Pydantic where appropriate
- clean module separation
- descriptive naming
- no giant functions
- minimal duplication
- deterministic configuration
- explicit error handling

Avoid unnecessary abstraction.

Do not introduce ten frameworks where standard Python is enough.

61. DEPENDENCY DISCIPLINE

Likely dependencies may include:

- pandas
- numpy
- scikit-learn
- imbalanced-learn
- matplotlib
- xgboost
- lightgbm
- catboost
- optuna
- shap
- lime
- FastAPI
- Pydantic
- pytest

Only install dependencies actually used.

Pin versions in a reproducible environment.

Before introducing another platform or library, explain why it is required.

62. GIT HYGIENE

Do not commit:

- datasets if licensing/size makes that inappropriate
- secrets
- AWS credentials
- Firebase keys
- virtual environments

- huge temporary model files unless intentionally managed
- local IDE caches

Update .gitignore.

Use clear commits by logical milestone.

Do not mix unrelated refactoring into Phase 1 research changes.

63. DATASET HASHING

Compute a SHA-256 hash of the exact dataset file used for final experiments.

Store it in experiment metadata.

This prevents a research-paper result from later being accidentally reproduced using a different dataset version.

64. EXPERIMENT IDENTITY

Every official experiment should receive an ID.

For example:

phase1_2026_08_23_seed42

Metadata should point to:

- dataset hash
- git commit

- config
 - split
 - models
 - tuning strategy
 - environment
 - results
-

65. NO SILENT SUBSAMPLING

Some models, particularly non-linear SVM or large LIME workloads, may become computationally expensive on 253k records.

If an experiment uses a sample:

- explicitly label it exploratory
- record sample size
- record sampling method
- do not compare sampled results directly with full-dataset final results as though protocols were identical

Do not silently reduce the dataset merely to make code finish.

66. PERFORMANCE PROFILING

For expensive operations record:

- wall-clock time
- peak memory where practical
- sample count
- hardware

This will help justify algorithm and deployment choices in the paper.

67. FIGURE QUALITY

Research figures must have:

- descriptive title
- axis labels
- readable legends
- model names
- correct units
- sufficient resolution
- reproducible generation code

Avoid screenshots of notebook output.

Save publication-quality PNG/PDF/SVG when supported.

68. ROC CURVE REQUIREMENT

Create one combined ROC comparison figure for the five Phase 1 models.

Also save individual ROC data.

The paper should explain:

- true-positive rate
- false-positive rate
- ROC-AUC

Do not state that AUC alone measures clinical utility.

69. CONFUSION MATRIX REQUIREMENT

For every Phase 1 model provide:

TN

FP

FN

TP

Explicitly calculate:

$\text{Sensitivity} = \text{TP} / (\text{TP} + \text{FN})$

$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$

This is particularly important because false negatives matter in screening.

70. PAPER RESULTS MUST TRACE TO ARTIFACTS

Every important research number should be traceable.

Example:

Paper says:

Random Forest recall = X

There should be a machine-readable artifact containing the same X.

Avoid manually calculated paper values.

71. SOURCE OF TRUTH FOR RESULTS

Use:

`results/phase1/model_comparison.csv`

as the canonical Phase 1 summary.

The paper, presentation, and README should derive their key values from it.

This prevents inconsistent metrics across deliverables.

72. PRESENTATION DEADLINE MODE

Because Phase 1 must be presented by the first week of September, prioritise work in this order:

Priority 1

Correct dataset pipeline.

Priority 2

Five reproducible models.

Priority 3

Real evaluation metrics.

Priority 4

ROC curves and comparison.

Priority 5

Initial SHAP/LIME.

Priority 6

Ethical-analysis section.

Priority 7

Paper draft.

Priority 8

Panel material.

Do not spend the critical Phase 1 window polishing the Flutter app before the five-model experimental results exist.

The mobile application is a final deliverable, but the immediate supervisor milestone is model comparison + paper + presentation.

73. CURRENT-PHASE DEFINITION OF DONE

Phase 1 is not "done" when code executes once.

It is done only when:

- ☐ dataset is validated
 - ☐ reproducible split exists
 - ☐ no leakage exists
 - ☐ five approved algorithms are trained
 - ☐ CV/tuning methodology is documented
 - ☐ held-out test evaluation completed
 - ☐ accuracy calculated
 - ☐ precision calculated
 - ☐ recall calculated
 - ☐ F1 calculated
 - ☐ specificity calculated
 - ☐ ROC-AUC calculated
 - ☐ ROC curves generated
 - ☐ confusion matrices generated
 - ☐ calibration assessed
 - ☐ results saved machine-readably
 - ☐ best-model tradeoffs documented
 - ☐ preliminary SHAP implemented
 - ☐ preliminary LIME implemented
 - ☐ SHAP/LIME disagreement discussed
 - ☐ ethical concerns written
 - ☐ limitations written
 - ☐ research paper draft created
 - ☐ presentation content generated
 - ☐ tests passing
 - ☐ README updated
 - ☐ no fake metrics anywhere
-

74. QUESTIONS THE PANEL SHOULD BE ABLE TO ASK

Build the project and documentation so the team can confidently answer:

Dataset

Why BRFSS instead of PIMA?

Why is US data being used for an Indian application?

What are the 21 features?

How imbalanced is the target?

Methodology

Why 70:15:15?

Why stratification?

Why SMOTE?

When exactly is SMOTE applied?

How did you avoid leakage?

Why 5-fold CV?

Models

Why these five?

Why eventually ten?

Which algorithm performed best?

Why?

Metrics

Why not accuracy alone?

What is recall?

What is F1?

What is ROC?

What does AUC mean?

What is a false negative?

Explainability

What is SHAP?

What is LIME?

How are they different?

Why use both?

What happens if they disagree?

Ethics

Can an explanation be misleading?

Does SHAP prove causality?

Can the system diagnose diabetes?

How will bias be addressed?

Why might results not transfer to India?

Deployment

Why Flutter?

Why FastAPI?

Why AWS?

Why Firebase?

How will user data be protected?

75. CURSOR EXECUTION BEHAVIOUR

Do not respond by merely writing a long plan and stopping.

Actually work through the repository.

Your sequence should be:

Step 1

Inspect files and code.

Step 2

Give a concise audit of what exists.

Step 3

Identify blocking ambiguity, especially which exact five Phase 1 algorithms are approved.

Step 4

Create/update the architecture and research-contract documentation.

Step 5

Implement the Phase 1 data pipeline.

Step 6

Run tests.

Step 7

Implement the model registry and training framework.

Step 8

Run tests.

Step 9

Configure the five Phase 1 models after selection is confirmed.

Step 10

Train/tune them.

Step 11

Generate real metrics and figures.

Step 12

Run the explainability pipeline.

Step 13

Generate the Phase 1 report.

Step 14

Create/update the research paper draft.

Step 15

Create the panel-presentation content.

Step 16

Report exactly:

Completed

Remaining

Experimental results

Failures

Risks

Decisions requiring confirmation

Next recommended action

76. DO NOT DO THESE THINGS

Do not:

- fabricate results
- train on the test set
- SMOTE before splitting
- tune on test data
- use test data to choose the model
- silently change the dataset
- silently subsample
- call SHAP causal
- call LIME causal
- call SHAP/LIME agreement proof of prediction correctness
- claim Indian validation before it exists
- call the app diagnostic
- generate unrestricted AI medical advice

- expose AWS root credentials
 - commit secrets
 - redesign approved architecture without justification
 - build the mobile UI before validating the core ML pipeline if doing so threatens the Phase 1 deadline
 - report accuracy alone
 - hardcode fake metrics into the paper
 - mark future features as already completed
 - overwrite existing project files blindly
 - create duplicate implementations when reusable infrastructure already exists
-

77. FIRST TASK TO PERFORM NOW

Start by examining the complete repository and all reference documents.

Then produce:

docs/source_of_truth.md

with the following sections:

1. Project objective
2. Final deliverables
3. Immediate Phase 1 deliverables
4. Dataset
5. Required Phase 1 metrics
6. Full 10-model candidate suite
7. Current five-model-selection status
8. Explainability requirements
9. Ethical requirements
10. Mobile architecture
11. Cloud/infrastructure guidance
12. Source discrepancies
13. Decisions requiring confirmation

Then immediately begin implementing the leakage-safe Phase 1 data pipeline and experiment framework.

Do not wait for the entire eight-month design to be perfect before coding.

However, do not run the final official five-model experiment until the exact five-model list is either:

- already explicitly defined in the project material, or
 - confirmed by the human user.
-

78. FINAL QUALITY STANDARD

The completed project should allow another technically competent researcher to clone the repository, obtain the permitted dataset, reproduce the experiment, regenerate the metrics, regenerate the plots, understand why the selected model was chosen, inspect the SHAP/LIME explanations, identify the system's ethical limitations, run the FastAPI service, and understand how the Flutter mobile client consumes the prediction.

The research paper must describe what was actually done.

The presentation must show what was actually measured.

The application must communicate what the model can and cannot claim.

Accuracy without explainability is insufficient for this project.

Explainability without methodological correctness is also insufficient.

The project must therefore optimise simultaneously for:

predictive performance + recall + calibration + reproducibility + explanation quality + ethical transparency + deployability.

Proceed carefully, verify every stage, keep experimental evidence reproducible, and never substitute plausible-looking output for real measured results.